GraphOne Intelligence Pipeline - Architecture

1. Architecture Diagram & Data Flow

LOCAL DEVELOPMENT vs PRODUCTION DEPLOYMENT

LOCAL: SQLite DB, asyncio.Queue, Single Machine Async Loop

PROD: PostgreSQL, RabbitMQ/Celery, Kubernetes Horizontal Scaling

---

Data Flow:

1. Source Ingestion: aiohttp & RSS fetchers pull raw data.
2. Async Workers: Placed in bounded queues.
3. Extraction: Deterministic regex/meta extraction prioritized.
4. Validation: Strict Pydantic models drop fabricated rows.
5. Entity Resolution: RapidFuzz normalises variations.
6. Storage: Idempotent Upserts into database via content_id hash.

GraphOne Intelligence Pipeline - Architecture

2. Reliability Mechanisms

- 429 Strategy: The HttpClient intercepts 429s, respects Retry-After headers, and triggers an exponential backoff (base * 2^attempt + jitter) to protect upstream APIs.
- 413 Strategy: The tiktoken engine safely chunks large text at semantic boundaries (paragraphs > spaces) to prevent LLM context-window exhaustion.
- LLM Fallback: The orchestrator cascades failures from Gemini -> Groq -> DeepSeek, assuming untrusted output and handling 503s seamlessly.
- Freshness: 24-hour exact boundary calculated natively in UTC from 7-tier date extraction layer.
- Provenance: Pydantic layers forcefully require valid, resolved source_urls to prevent hallucinated URLs.
- Deduplication: SHA-256 hash of canonical URL serves as the content_id Primary Key to ensure idempotent writes.

GraphOne Intelligence Pipeline - Architecture

3. 500k+ Production Scaling Strategy

To scale to 500,000+ records, the architecture requires no rewrite, only infrastructure swapping:

- Worker Scaling: The asyncio.Queue is replaced by a distributed message broker (Kafka/RabbitMQ) and workers scale horizontally via Celery pods.
- PostgreSQL: SQLite is swapped for Postgres. The strict B-tree unique index on content_id natively manages thousands of concurrent upsert requests safely.
- Connection Pooling: pgbouncer sits in front of Postgres to mux thousands of async SQLAlchemy requests into minimal DB connections.
- Observability: Structured JSON logging is piped to Datadog/ELK, aggregating worker failures natively without blocking execution.
- Failure Isolation: Crawler nodes, LLM chunker nodes, and database writer nodes operate in disjoint deployments, allowing decoupled auto-scaling.